

Variational Autoencoder (VAE) + Latent Space Visualization

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
import matplotlib.pyplot as plt

# =====
# DATA
# =====
transform = transforms.ToTensor()
train_loader = torch.utils.data.DataLoader(
    datasets.MNIST('./data', train=True, download=True,
transform=transform),
    batch_size=128, shuffle=True
)

# =====
# VAE MODEL
# =====
class VAE(nn.Module):
    def __init__(self):
        super().__init__()

        self.fc1 = nn.Linear(784, 400)
        self.fc_mu = nn.Linear(400, 2)          # latent dim = 2
        self.fc_logvar = nn.Linear(400, 2)

        self.fc3 = nn.Linear(2, 400)
        self.fc4 = nn.Linear(400, 784)

    def encode(self, x):
        h = torch.relu(self.fc1(x))
        return self.fc_mu(h), self.fc_logvar(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        h = torch.relu(self.fc3(z))
        return torch.sigmoid(self.fc4(h))

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar
```

```

model = VAE()
optimizer = optim.Adam(model.parameters(), lr=1e-3)

# =====
# LOSS
# =====
def loss_fn(recon, x, mu, logvar):
    BCE = nn.functional.binary_cross_entropy(recon, x,
reduction='sum')
    KL = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return BCE + KL

# =====
# TRAIN
# =====
for epoch in range(5):
    total_loss = 0
    for x, _ in train_loader:
        x = x.view(-1, 784)

        recon, mu, logvar = model(x)
        loss = loss_fn(recon, x, mu, logvar)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch}, Loss: {total_loss}")

# =====
# LATENT SPACE VISUALIZATION
# =====
import numpy as np

z_list = []
labels = []

with torch.no_grad():
    for x, y in train_loader:
        x = x.view(-1, 784)
        mu, _ = model.encode(x)
        z_list.append(mu)
        labels.append(y)

z = torch.cat(z_list).numpy()
labels = torch.cat(labels).numpy()

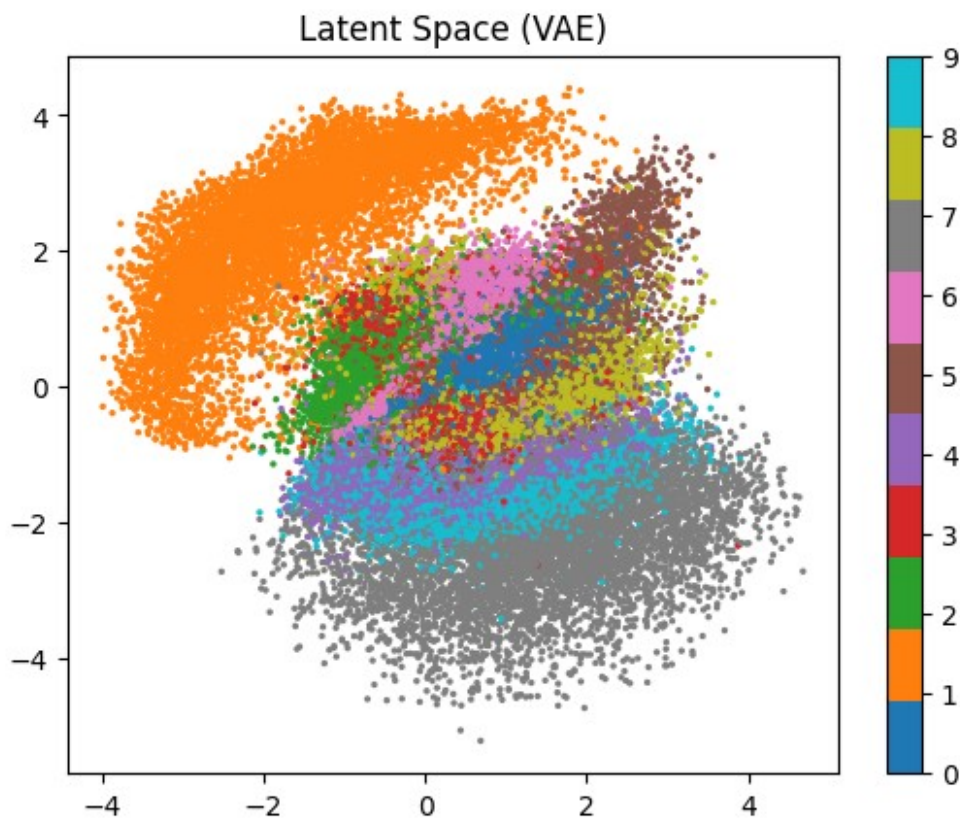
plt.scatter(z[:,0], z[:,1], c=labels, cmap='tab10', s=2)

```

```
plt.colorbar()
plt.title("Latent Space (VAE)")
plt.show()
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 42.7MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 1.07MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 9.82MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 7.60MB/s]
```

```
Epoch 0, Loss: 11414876.419921875
Epoch 1, Loss: 10117378.293945312
Epoch 2, Loss: 9823914.620117188
Epoch 3, Loss: 9644572.3125
Epoch 4, Loss: 9529755.69140625
```



GAN for Image Synthesis

```
import torch
import torch.nn as nn

# =====
# GENERATOR
# =====
class Generator(nn.Module):
```

```

def __init__(self):
    super().__init__()
    self.net = nn.Sequential(
        nn.Linear(100, 256),
        nn.ReLU(),
        nn.Linear(256, 784),
        nn.Tanh()
    )

def forward(self, z):
    return self.net(z)

# =====
# DISCRIMINATOR
# =====
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(784, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.net(x)

G = Generator()
D = Discriminator()

criterion = nn.BCELoss()
opt_G = torch.optim.Adam(G.parameters(), lr=0.0002)
opt_D = torch.optim.Adam(D.parameters(), lr=0.0002)

# =====
# TRAIN
# =====
for epoch in range(5):
    for x, _ in train_loader:
        x = x.view(-1, 784)

        # REAL
        real_labels = torch.ones(x.size(0), 1)
        fake_labels = torch.zeros(x.size(0), 1)

        # =====
        # TRAIN D
        # =====
        z = torch.randn(x.size(0), 100)

```

```

fake = G(z)

loss_real = criterion(D(x), real_labels)
loss_fake = criterion(D(fake.detach()), fake_labels)
loss_D = loss_real + loss_fake

opt_D.zero_grad()
loss_D.backward()
opt_D.step()

# =====
# TRAIN G
# =====
loss_G = criterion(D(fake), real_labels)

opt_G.zero_grad()
loss_G.backward()
opt_G.step()

print(f"Epoch {epoch} | D Loss: {loss_D.item()} | G Loss:
{loss_G.item()}")

```

GCN on Citation Data (Cora Dataset)

```
pip install torch-geometric
```

```
Collecting torch-geometric
```

```
  Downloading torch_geometric-2.7.0-py3-none-any.whl.metadata (63 kB)
  63.7/63.7 kB 2.2 MB/s eta
```

```
0:00:00
```

```
Requirement already satisfied: aiohttp in /usr/local/lib/python3.12/dist-
packages (from torch-geometric) (3.13.3)
```

```
Requirement already satisfied: fsspec in
/usr/local/lib/python3.12/dist-packages (from torch-geometric)
(2026.2.0)
```

```
Requirement already satisfied: jinja2 in
/usr/local/lib/python3.12/dist-packages (from torch-geometric) (3.1.6)
```

```
Requirement already satisfied: numpy in
/usr/local/lib/python3.12/dist-packages (from torch-geometric) (2.0.2)
```

```
Requirement already satisfied: psutil>=5.8.0 in
/usr/local/lib/python3.12/dist-packages (from torch-geometric) (5.9.5)
```

```
Requirement already satisfied: pyparsing in
/usr/local/lib/python3.12/dist-packages (from torch-geometric) (3.3.2)
```

```
Requirement already satisfied: requests in
/usr/local/lib/python3.12/dist-packages (from torch-geometric)
(2.32.4)
```

```
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-
packages (from torch-geometric) (4.67.3)
```

```
Requirement already satisfied: xxhash in
/usr/local/lib/python3.12/dist-packages (from torch-geometric) (3.6.0)
```

```
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch-
geometric) (2.6.1)
Requirement already satisfied: aiosignal>=1.4.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch-
geometric) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch-
geometric) (25.4.0)
Requirement already satisfied: frozenlist>=1.1.1 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch-
geometric) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch-
geometric) (6.7.1)
Requirement already satisfied: propcache>=0.2.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch-
geometric) (0.4.1)
Requirement already satisfied: yarll<2.0,>=1.17.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch-
geometric) (1.22.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch-geometric)
(3.0.3)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests->torch-
geometric) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.12/dist-packages (from requests->torch-
geometric) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests->torch-
geometric) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests->torch-
geometric) (2026.1.4)
Requirement already satisfied: typing-extensions>=4.2 in
/usr/local/lib/python3.12/dist-packages (from aiosignal>=1.4.0-
>aiohttp->torch-geometric) (4.15.0)
Downloading torch_geometric-2.7.0-py3-none-any.whl (1.3 MB)
----- 1.3/1.3 MB 24.2 MB/s eta
0:00:00a 0:00:01
etric
Successfully installed torch-geometric-2.7.0
Note: you may need to restart the kernel to use updated packages.
```

```
import torch
import torch.nn.functional as F
from torch_geometric.datasets import Planetoid
from torch_geometric.nn import GCNConv
```

```

# =====
# LOAD DATA
# =====
dataset = Planetoid(root='data', name='Cora')
data = dataset[0]

# =====
# MODEL
# =====
class GCN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = GCNConv(dataset.num_features, 16)
        self.conv2 = GCNConv(16, dataset.num_classes)

    def forward(self, data):
        x, edge_index = data.x, data.edge_index

        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = self.conv2(x, edge_index)

        return F.log_softmax(x, dim=1)

model = GCN()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

# =====
# TRAIN
# =====
for epoch in range(200):
    model.train()
    optimizer.zero_grad()

    out = model(data)
    loss = F.nll_loss(out[data.train_mask], data.y[data.train_mask])

    loss.backward()
    optimizer.step()

    if epoch % 20 == 0:
        print(f"Epoch {epoch}, Loss: {loss.item()}")

# =====
# TEST
# =====
model.eval()
pred = model(data).argmax(dim=1)

```

```
correct = (pred[data.test_mask] == data.y[data.test_mask]).sum()  
acc = int(correct) / int(data.test_mask.sum())
```

```
print(f"Test Accuracy: {acc:.4f}")
```

Downloading

<https://github.com/kimiyoung/planetoid/raw/master/data/ind.cora.x>

Downloading

<https://github.com/kimiyoung/planetoid/raw/master/data/ind.cora.tx>

Downloading

<https://github.com/kimiyoung/planetoid/raw/master/data/ind.cora.allx>

Downloading

<https://github.com/kimiyoung/planetoid/raw/master/data/ind.cora.y>

Downloading

<https://github.com/kimiyoung/planetoid/raw/master/data/ind.cora.ty>

Downloading

<https://github.com/kimiyoung/planetoid/raw/master/data/ind.cora.ally>

Downloading

<https://github.com/kimiyoung/planetoid/raw/master/data/ind.cora.graph>

Downloading

<https://github.com/kimiyoung/planetoid/raw/master/data/ind.cora.test.index>

Processing...

Done!

Epoch 0, Loss: 1.9524590969085693

Epoch 20, Loss: 0.09654643386602402

Epoch 40, Loss: 0.008083602413535118

Epoch 60, Loss: 0.0033606276847422123

Epoch 80, Loss: 0.002299909945577383

Epoch 100, Loss: 0.0017840188229456544

Epoch 120, Loss: 0.0014446248533204198

Epoch 140, Loss: 0.0011969482293352485

Epoch 160, Loss: 0.0010085850954055786

Epoch 180, Loss: 0.0008617500425316393

Test Accuracy: 0.7800